

FLOWDESK

Enterprise Workflow & Approval Management Platform

Product Requirements Document (PRD) - Version 1.0

Production-Ready Product & Engineering Specification

- **Primary Backend:** ASP.NET Core / C#
- **Database:** SQL Server
- **Frontend:** Next.js / React / TypeScript
- **Architecture:** Clean Architecture + CQRS
- **AI:** OpenRouter Final Phase Only

1. Executive Summary

FlowDesk is a secure enterprise workflow and approval management platform that digitizes internal business requests and routes them through configurable approval processes. The platform is intentionally not AI-first. Its primary value is deterministic workflow automation, authorization, auditability, reliability, and operational visibility.

1.1 Product Vision

Provide organizations with a centralized system for creating, routing, approving, tracking, and auditing business requests without hard-coding every business process into the application.

1.2 Product Principles

- Security is a first-class requirement from the beginning, not a final-phase feature.
- Workflow behavior must be deterministic, testable, versioned, and auditable.
- AI is optional and must never become a dependency for core business operations.
- Every important business action must be traceable.
- Production readiness includes testing, observability, deployment, backups, recovery, and operational controls.
- Prefer simple, maintainable solutions over unnecessary architectural complexity.

2. Goals & Non-Goals

2.1 Goals

- Build a realistic enterprise-grade ASP.NET Core backend.
- Implement configurable workflows and dynamic approval routing.
- Implement secure authentication, authorization, RBAC, and permission-based access.
- Implement complete audit trails and request history.
- Implement background processing, notifications, reminders, SLA tracking, and escalation.
- Provide production-grade testing, logging, health checks, CI/CD, and deployment.
- Use Redis and other infrastructure only where justified by measurable requirements.
- Introduce lightweight OpenRouter text features only after the core product is production-ready.

2.2 Non-Goals

- A complete ERP or accounting platform.
- A full HR management system.
- A general-purpose BPMN suite.
- An AI-agent platform.
- AI-controlled approvals, permissions, or security decisions.
- A payment-processing platform.

3. Core Business Model

The central business object is a Request. Request Types can include Purchase, Expense, Leave, Asset, IT Access, Maintenance, Payment, Document Approval, and General Business Request.

3.1 Example Workflow

Purchase $\leq$ 5,000 EGP : Employee -> Manager -> Completed

Purchase $>$ 5,000 EGP : Employee -> Manager -> Finance -> Completed

Purchase $>$ 50,000 EGP : Employee -> Manager -> Finance -> Procurement -> Completed

The routing decision must be performed by the workflow engine using configured rules rather than hard-coded controller logic.

4. Users, Roles & Permissions

Role	Primary Responsibilities
Super Admin	Organizations, users, roles, permissions, workflows, system settings, audit and system reports.
Organization Admin	Users, departments, positions, request types, workflows, organization settings and reports.
Employee	Create and manage own requests, comments, attachments and returned requests.
Manager	Process assigned approvals, approve/reject/return, comment and delegate.
Finance Officer	Process financial approvals according to assigned workflow rules.
Procurement Officer	Process procurement-related approvals according to workflow rules.

4.1 Permission Model

Purchase.Create, Purchase.Read, Purchase.Update, Purchase.Approve, Purchase.Reject, Workflow.Manage, Workflow.Publish, Users.Manage, Reports.Read, AuditLogs.Read

5. Authentication & Authorization

- JWT access tokens with short lifetimes and refresh tokens with rotation and revocation.
- Secure password hashing; never store plaintext passwords.
- Password reset and account recovery flows using short-lived, single-use tokens.
- Account logout / throttling for repeated failed authentication attempts.
- Server-side authorization for every protected operation; frontend checks are never sufficient.
- Support role-based and permission-based authorization, with resource-level checks for request ownership and approval responsibility.
- HTTPS in production, strict CORS, security headers, rate limiting, request validation, and secure error responses.
- Never expose secrets, stack traces, database errors, internal paths, or tokens in API responses.

6. Request Lifecycle

Draft -> Submitted -> PendingApproval

|-> Returned -> Draft -> Resubmitted -> PendingApproval

|-> Rejected

|-> Cancelled

|-> Approved -> Completed

Implement a controlled state machine. Invalid transitions must be rejected at the application/domain level and cannot be bypassed by direct API calls.

7. Workflow Engine

The workflow engine is the core technical capability. A Workflow contains versioned steps, conditions, approver definitions, and execution rules.

- Workflow definitions must be separated from workflow instances.
- Published workflow versions are immutable.
- New workflow changes create a new version.

- Existing requests continue using the workflow version under which they started.
- Support sequential approval steps and multiple approvers where required.
- Approvers can be resolved from users, roles, departments, positions, or organizational relationships.
- Workflow execution must be deterministic and fully auditable.

7.1 Initial Condition Engine

Supported operators: Equals, NotEquals, GreaterThan, GreaterThanOrEqual, LessThan, LessThanOrEqual, Contains, In

Example: Amount > 10,000 AND Department == "IT" AND RequestType == "Purchase" AND Priority == "High"

8. Approval System

- Create approval instances from workflow steps.
- Support Pending, Approved, Rejected, Returned, Cancelled, Delegated, and Expired states.
- Record every approval action with actor, timestamp, decision, comment, and request/workflow context.
- Prevent duplicate or invalid approvals through state validation, transactions, concurrency controls, and idempotency where appropriate.
- Returned requests must be editable only within the rules defined by the workflow.

9. Delegation, SLA & Escalation

9.1 Delegation

- Allow users to delegate approvals for a defined time window.
- Validate date range, permissions, self-delegation, overlapping delegations, and circular delegation.
- Delegation must be auditable.

9.2 SLA & Escalation

Approval SLA: 48 hours -> 24h: Reminder -> 48h: Overdue -> 72h: Escalation

Escalation targets may include a manager's manager, department manager, or organization administrator, according to configuration.

10. Audit Trail & Compliance

- Record request creation, submission, workflow start, assignment, approvals, rejections, returns, resubmissions, comments, attachments, cancellation, and completion.
- Record authentication and security events such as login failures, password changes, token revocation, and permission changes.
- Audit records should contain user, action, entity, entity ID, timestamp, IP address where appropriate, user agent, and relevant old/new values.
- Audit logs are append-only from the application perspective; normal users cannot edit or delete them.
- Sensitive values must be redacted or excluded from logs.

11. Comments & Attachments

11.1 Comments

- Chronological request comments.
- Author and timestamp.
- Comments included in request history where appropriate.

11.2 Secure Attachments

- Validate extension and MIME type.
- Enforce file size limits.
- Generate server-side filenames.
- Prevent executable or dangerous file uploads.
- Store files outside executable directories.

- Authorize every download.
- Record upload/download/deletion events where required.
- Use malware scanning where supported by deployment infrastructure.

12. Notifications & Background Processing

Implement an abstraction such as `INotificationService` so notification providers remain replaceable.

- In-app notifications.
- Email notifications.
- Approval required notifications.
- Approval result notifications.
- Return/rejection/completion notifications.
- Reminder and escalation notifications.
- Notification templates.

12.1 Background Jobs

`SendNotification`, `SendApprovalReminder`, `CheckExpiredApprovals`, `ProcessEscalations`, `GenerateScheduledReports`, `CleanupTemporaryFiles`

Use Hangfire or an equivalent job system. Jobs must be observable, retryable, and idempotent where necessary. Failed jobs must be visible to operators.

13. Concurrency & Idempotency

- Use optimistic concurrency for requests and approval instances.
- Protect critical state transitions with transactions and database constraints.
- Prevent duplicate approvals caused by retries or double-clicks.
- Use idempotency keys for operations where duplicate execution could have material consequences.
- Return appropriate conflict responses when concurrent updates cannot be safely merged.

14. Database Requirements

Users, Roles, Permissions, UserRoles, RolePermissions, Organizations, Departments, Positions, RequestTypes, Requests, RequestItems, Workflows, WorkflowVersions, WorkflowSteps, WorkflowConditions, ApprovalInstances, ApprovalActions, Comments, Attachments, Notifications, Delegations, AuditLogs, RefreshTokens

- Use EF Core migrations.
- Maintain referential integrity with foreign keys.
- Create indexes based on actual query patterns.
- Avoid unnecessary denormalization.
- Use transactions for multi-record business operations.
- Do not expose EF Core entities directly from API contracts.

15. Backend Architecture

```
src/  
  Domain/  
  Application/  
  Infrastructure/  
  API/  
Tests/  
  UnitTests/  
  IntegrationTests/  
  ArchitectureTests/
```

- **Domain:** entities, value objects, domain events, enums, and business rules.
- **Application:** commands, queries, DTOs, validators, interfaces, authorization policies, and use cases.
- **Infrastructure:** EF Core, SQL Server, Redis, email, storage, Hangfire, and external providers.
- **API:** HTTP endpoints, middleware, authentication, exception handling, OpenAPI/Swagger, and configuration.

16. API Standards

```
GET /api/v1/requests
GET /api/v1/requests/{id}
POST /api/v1/requests
PUT /api/v1/requests/{id}
DELETE /api/v1/requests/{id}
POST /api/v1/requests/{id}/submit
POST /api/v1/requests/{id}/approve
POST /api/v1/requests/{id}/reject
POST /api/v1/requests/{id}/return
POST /api/v1/requests/{id}/cancel
GET /api/v1/workflows
POST /api/v1/workflows
PUT /api/v1/workflows/{id}
POST /api/v1/workflows/{id}/publish
```

Use consistent response and validation formats, server-side pagination/filtering/sorting, proper HTTP status codes, cancellation tokens, and API versioning.

17. Performance & Caching

- Pagination by default for large collections.
- Projection and efficient querying instead of loading unnecessary entity graphs.
- Avoid N+1 queries.
- Use asynchronous database access.
- Profile slow queries and add indexes based on evidence.
- Introduce Redis for relatively stable data such as permissions, workflow definitions, request types, or organization settings where caching is justified.
- Implement explicit cache expiration and invalidation.
- The system should degrade gracefully if Redis is unavailable where practical.

18. Frontend Requirements

- Next.js, React, TypeScript, Tailwind CSS, and TanStack Query or equivalent.
- Responsive layouts for desktop, tablet, and mobile.
- Role-aware navigation and server-enforced authorization.
- Loading, error, empty, validation, and success states for every major workflow.
- Request list and details screens.
- Approval inbox and approval details.
- Workflow administration screens.
- Users, roles, permissions, departments, notifications, reports, audit logs, and settings.

19. Dashboards & Reporting

19.1 Employee Dashboard

My Requests | Pending | Approved | Rejected | Returned

19.2 Manager Dashboard

Pending Approvals | Approved Today | Rejected | Overdue

19.3 Admin Dashboard

Total Requests | Pending | Average Approval Time | Department Statistics | Request Types | SLA Compliance | Workflow Bottlenecks

19.4 Search & Filtering

Support server-side filtering by request number, type, status, requester, department, date range, priority, amount, and approver.

20. Observability & Operations

- Use structured logging such as Serilog.
- Include correlation IDs, request IDs, user IDs where appropriate, endpoint, duration, and status code.
- Implement /health, /health/live, and /health/ready or equivalent.
- Monitor database, Redis, background jobs, and external dependencies.

- Do not expose sensitive infrastructure details through health endpoints.
- Provide operational visibility into failed background jobs.

21. Testing Strategy

- Unit tests for domain rules, workflow transitions, conditions, approval logic, delegation, SLA calculations, and authorization policies.
- Integration tests covering API-to-database flows and critical business scenarios.
- Security tests for unauthorized access, horizontal/vertical privilege escalation, IDOR, token expiry, and attachment authorization.
- Architecture tests to enforce Clean Architecture dependency boundaries.
- Regression tests for previously fixed production defects.
- Critical workflow paths must have automated coverage before production release.

22. CI/CD & Deployment

Push -> Restore -> Build -> Lint / Static Checks -> Unit Tests -> Integration Tests -> Security Checks -> Docker Build -> Deploy to Staging -> Smoke Tests -> Production Release

- Use GitHub Actions or equivalent.
- Production deployment must be gated by successful validation.
- Maintain Development, Staging, and Production configurations.
- Use environment variables or a secret manager for credentials and API keys.
- Provide Dockerfile and development docker-compose configuration.
- Document rollback procedures.

23. Backup & Disaster Recovery

- Automated SQL Server backups in production.
- Defined retention policy.
- Restore testing at planned intervals.
- Documented recovery procedure.

- Backups must be stored separately from the primary application environment where appropriate.
- A backup is not considered verified until a restore has been successfully tested.

24. Production Security Checklist

- HTTPS enforced.
- Secure authentication and token rotation.
- Strict authorization on every protected endpoint.
- Rate limiting.
- Secure CORS configuration.
- Security headers.
- Input validation.
- Secure file upload and download authorization.
- Secrets externalized.
- Sensitive logs redacted.
- Database credentials follow least privilege.
- No debug mode or verbose exception details in production.
- Dependency and container vulnerability scanning where available.
- Database backup and restore process verified.

25. Development Phases

Phase 0: Architecture & Foundation

- **Scope:** Repository and solution setup; Clean Architecture; project boundaries; configuration; logging; exception handling; API versioning; Swagger; Docker development environment; SQL Server; EF Core; health checks; Git workflow; initial CI pipeline.
- **Deliverable:** A clean application that builds, runs, logs correctly, connects to SQL Server, and passes CI.

Phase 1: Identity & Security

- **Scope:** Authentication; JWT; refresh tokens; revocation; password hashing; reset; lockout; roles; permissions; authorization policies; security baseline review.
- **Deliverable:** Secure authentication and authorization foundation.

Phase 2: Organization Management

- **Scope:** Organizations; departments; positions; users; managers; hierarchy; role assignment; permissions.
- **Deliverable:** Organization structure available to workflow resolution.

Phase 3: Request Management

- **Scope:** Request types; drafts; create/edit/submit/cancel; request numbering; comments; attachments; history; search; filters; pagination.
- **Deliverable:** Functional request management module.

Phase 4: Approval Engine

- **Scope:** Approval instances; approve/reject/return; comments; sequential approvals; state machine; transactions; concurrency; idempotency.
- **Deliverable:** Reliable approval engine.

Phase 5: Workflow Engine

- **Scope:** Workflow definitions; steps; conditions; approver resolution; publishing; versioning; activation; validation.
- **Deliverable:** Configurable workflows without hard-coded business processes.

Phase 6: Delegation, SLA & Escalation

- **Scope:** Delegation; SLA configuration; reminders; overdue detection; escalation rules and notifications.
- **Deliverable:** Enterprise workflow controls.

Phase 7: Notifications & Background Processing

- **Scope:** In-app and email notifications; Hangfire; retries; recurring jobs; templates; failure visibility.
- **Deliverable:** Reliable asynchronous processing.

Phase 8: Audit & Compliance

- **Scope:** Business audit trail; security events; entity history; immutable audit records.
- **Deliverable:** Full traceability.

Phase 9: Redis & Performance Optimization

- **Scope:** Query profiling; indexing; Redis; cache policies; invalidation; performance benchmarks.
- **Deliverable:** Measured performance improvements.

Phase 10: Reporting & Analytics

- **Scope:** Request and approval statistics; processing time; SLA compliance; rejection rate; department and workflow analytics.
- **Deliverable:** Management reporting and dashboards.

Phase 11: Production Hardening

- **Scope:** Security review; performance review; reliability review; observability review; backup/recovery; comprehensive automated testing; release readiness.
- **Deliverable:** Production release candidate.

Phase 12: Deployment & Production Release

- **Scope:** Production Docker image; staging; production environment; secrets; HTTPS; monitoring; migrations; backups; restore verification; smoke tests; rollback procedure.
- **Deliverable:** Live production deployment.

Phase 13: Lightweight AI Integration

- **Scope:** OpenRouter integration; text summary; description improvement; email generation; optional text classification; provider abstraction; timeout/retry/fallback/cost controls.
- **Deliverable:** Optional AI productivity layer that cannot block or control core business workflows.

26. AI Requirements (Final Phase Only)

AI must be introduced only after the core system has passed production hardening and release gates. The application must remain fully functional if the AI provider is unavailable.

`ITextGenerationService`

+-- OpenRouter Provider

| +-- Timeout / Retry / Rate Limit / Fallback

- Generate a short professional request summary.
- Improve a short request description.
- Generate professional notification/email text.
- Optionally classify request text into a known category.
- AI output must never directly approve/reject a request or modify authorization.
- Minimize data sent to the provider and exclude secrets, credentials, tokens, and unnecessary sensitive information.
- Implement timeouts, bounded retries, provider failure handling, usage/cost controls, and safe logging.

27. Release Gates

Core Features -> Security -> Automated Testing -> Performance -> Observability -> CI/CD -> Backup & Recovery -> Production Deployment

Only after these gates pass: Final Phase AI Integration

28. Definition of Done

- Business logic implemented in the correct architectural layer.
- Input validation implemented.
- Authorization implemented and tested.
- Error handling implemented.
- Audit implications considered.
- Logging and observability implemented where appropriate.
- Unit tests added.
- Integration tests added for critical paths.
- Database migration included.
- API documented.
- Frontend states completed.
- Security considerations reviewed.
- Performance implications considered.
- No known critical security or data-integrity defects.

29. Engineering Rules

29.1 Do Not

- Put business logic inside controllers.
- Expose EF Core entities directly through APIs.
- Hard-code workflow rules.
- Hard-code secrets.
- Trust frontend authorization.
- Ignore concurrency.
- Swallow exceptions.
- Log sensitive information.
- Add infrastructure technologies without a real requirement.
- Introduce AI before production hardening.

29.2 Do

- Keep controllers/endpoints thin.
- Use DTOs and explicit API contracts.
- Validate all external input.
- Use cancellation tokens for async operations.
- Use transactions for multi-step state changes.
- Apply least privilege.
- Write automated tests.
- Use structured logs and correlation IDs.
- Document important architecture decisions.
- Measure performance before optimizing.
- Keep the core business workflow deterministic.

30. Final Technology Stack

Layer	Technology
Backend	ASP.NET Core, C#, EF Core, SQL Server
Architecture	Clean Architecture, CQRS, MediatR
Validation	Fluent Validation
Mapping	AutoMapper or explicit mapping where preferable
Authentication	JWT + Refresh Tokens
Logging	Serilog
Background Jobs	Hangfire
Caching	Redis
API Documentation	Swagger/OpenAPI
Frontend	Next.js, React, TypeScript, Tailwind CSS, TanStack Query
Infrastructure	Docker, GitHub Actions
AI	OpenRouter - final phase only

31. Final Product Definition

FlowDesk should ultimately be presented as a production-ready enterprise workflow and approval platform built with ASP.NET Core, featuring configurable workflows, dynamic approval routing, RBAC, audit trails, SLA management, background processing, notifications, caching, observability, automated testing, CI/CD, secure deployment, and optional AI-powered text assistance.

The project is intentionally structured so that the business idea remains understandable while the engineering depth grows phase by phase. The final result should demonstrate practical backend engineering rather than simply CRUD implementation.